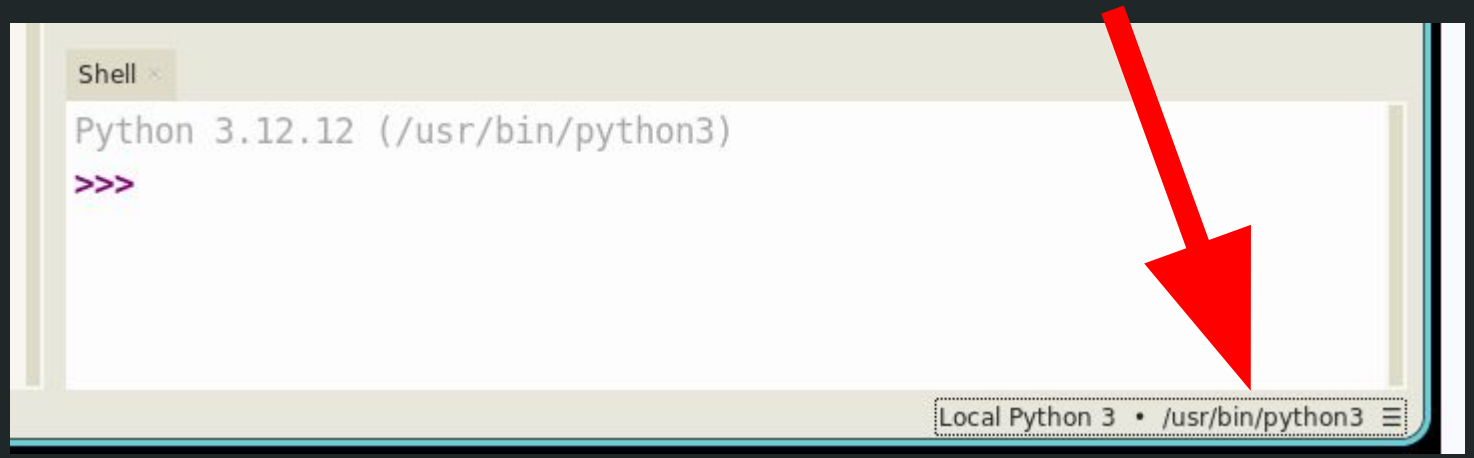
BBC micro:bit - Using the REPL

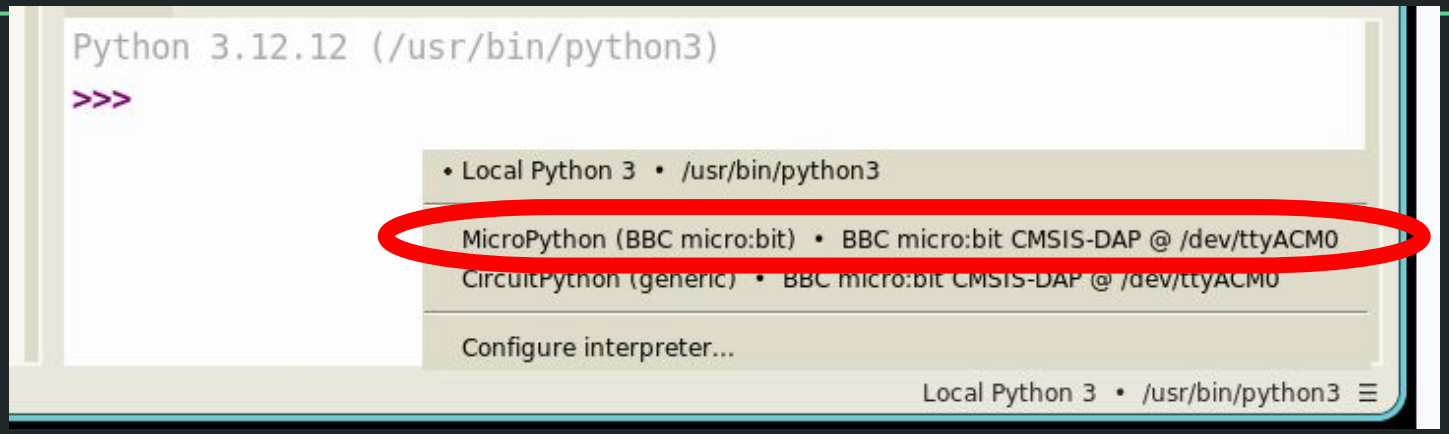
Steve Case

BBC micro:bit v2 Development Board

BBC micro:bit

Change to connect to micro:bit





BBC micro:bit

REPL is here >>>

File Edit View Run Tools Help

Files

This computer
/home/steve/microbit/
students

- robot.py
- sound2.py
- speech.py
- temperature1.py
- temperature2.py
- weather_station.py
- while_loop1.py

micro:bit

- 8ball_1.py
- disp_n.py
- eight_ball.py
- for_loop1.py
- main.py

dice.py | [main.py]

```
1  #
2  #  For loop demo with display
3  #
4
5  from microbit import *
6  import random
7  display.clear()
8
9
10 while Tru
11     1 in range(0, 5):
```

Shell | ption | Program tree | TODO

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Traceback (most recent call last):
  File "<stdin>", line 20, in <module>
KeyboardInterrupt:

>>>
```

MicroPython (BBC micro:bit) • BBC micro:bit CMSIS-DAP @ /dev/ttyACM0

BBC micro:bit

Thonny Python Editor

File Edit View Run Tools Help

Program tabs

Files

This computer
/home/steve/microbit/students

- robot.py
- sound2.py
- speech.py
- temperature1.py
- temperature2.py
- weather_station.py
- while_loop1.py

micro:bit

- 8ball_1.py
- disp_n.py
- eight_ball.py
- for_loop1.py
- main.py

dice.py | [main.py]

```
1  #
2  #  For loop demo with display
3  #
4
5  from microbit import *
6  import random
7  display.clear()
8
9
10 while True:
11     for i in range(0, 5):
```

Shell | Exception | Program tree | TODO

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Traceback (most recent call last):
  File "<stdin>", line 20, in <module>
KeyboardInterrupt:
>>>
```

REPL

MicroPython (BBC micro:bit) • BBC micro:bit CMSIS-DAP @ /dev/ttyACM0

REPL

REPL pronounced “rep-l” stands for

READ
EVALUATE
PRINT
LOOP

The REPL is used to communicate like a command-line to the Python interpreter.

We use it as a short-cut to test code pieces quickly before we start writing a program.

REPL

```
Explore:
Type 'help(something)' to find out about it. Type 'dir(something)' to see what
it can do. Type 'dir()' to see what stuff is available. For goodness sake,
don't type 'import this'.

Control commands:
 CTRL-C        -- stop a running program
 CTRL-D        -- on a blank line, do a soft reset of the micro:bit
 CTRL-E        -- enter paste mode, turning off auto-indent

For a list of available modules, type help('modules')

For more information about Python, visit: http://python.org/
To find out about MicroPython, visit: http://micropython.org/
Python/micro:bit documentation is here:
https://microbit-micropython.readthedocs.io/
>>>
```

REPL

```
>>> help()
Welcome to MicroPython on the micro:bit!

Try these commands:
 from microbit import *

 display.scroll('Hello')
 running_time()
 sleep(1000)
 button_a.is_pressed()

HINT: use the up and down arrow keys to get your command history.
Press the TAB key to auto-complete unfinished words
(so 'di' becomes 'display' after you press TAB).
```

Python Basics in the REPL- Variables

```
>>> i = 1
>>> print(i)
1
```

In this case, i is an integer. Think of integers as counting numbers.

```
>>> j = 2.
>>> print(j)
2.0
```

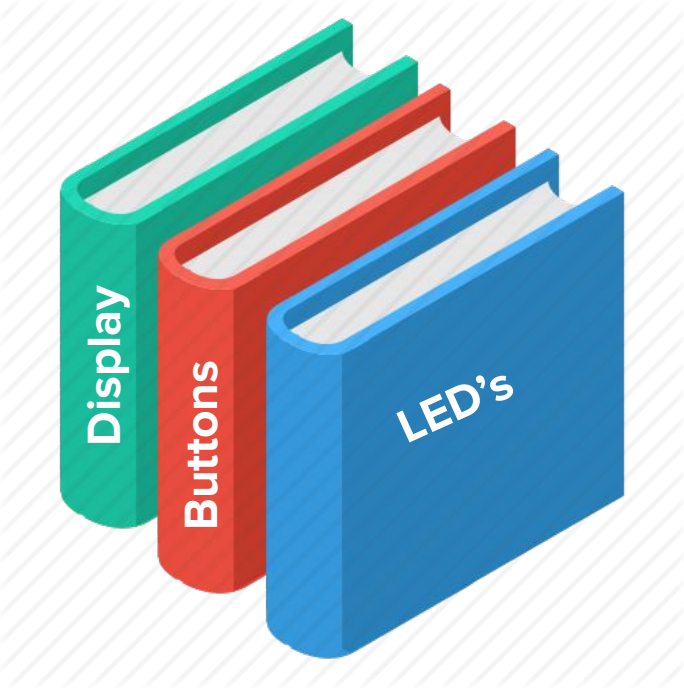
In this case, j is an floating point number. Think of it as a number with a fractional part.

```
>>> name = "Coco"
>>> print(name)
Coco
```

In this case, name is a string of collected characters.

Python Basics in the REPL- Libraries

- Libraries are collections of programs written by others, that make things easier to duplicate.
- A library can hold either one or many routines that we call by name inside them.
- When we want to use a library, we need to “check it out” and bring it into our program.

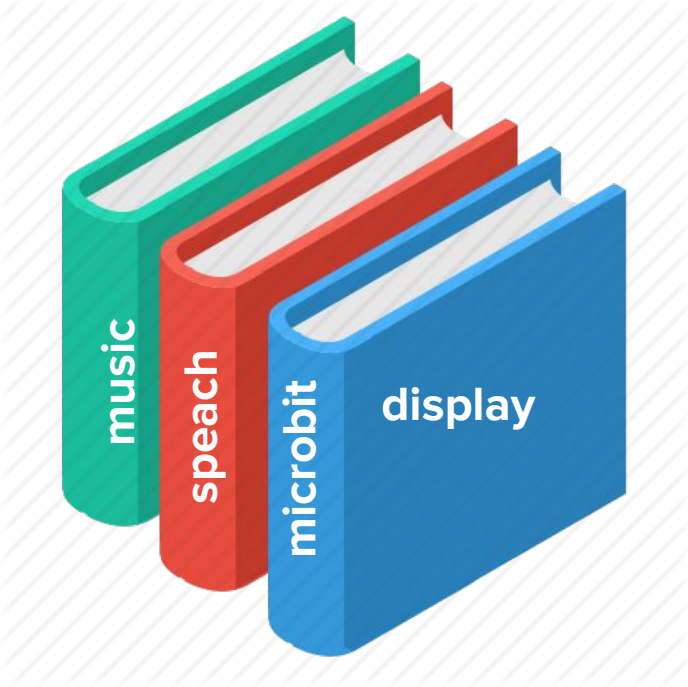


Python Basics in the REPL- Libraries

```
>>> from microbit import *
>>> display.scroll("Hi you!")
```

```
>>> import microbit
>>> microbit.display.scroll("Hi you!")
```

```
>>> from microbit import display
>>> microbit.display.scroll("Hi you!")
```



Images - import microbit, dir(Image)

'ALL_ARROWS', 'ALL_CLOCKS', 'ANGRY', 'ARROW_E', 'ARROW_N', 'ARROW_NE', 'ARROW_NW', 'ARROW_S', 'ARROW_SE', 'ARROW_SW', 'ARROW_W', 'ASLEEP', 'BUTTERFLY', 'CHESSBOARD', 'CLOCK1', 'CLOCK10', 'CLOCK11', 'CLOCK12', ' CLOCK2', 'CLOCK3', 'CLOCK4', CLOCK5', 'CLOCK6', CLOCK7', 'CLOCK8', CLOCK9', 'CONFUSED', 'COW', 'DIAMOND', 'DIAMOND_SMALL', 'DUCK', 'FABULOUS', 'GHOST', 'GIRAFFE', 'HAPPY', 'HEART', 'HEART_SMALL', 'HOUSE', 'MEH', 'MUSIC_CROTCHET', 'MUSIC_QUAVER', 'MUSIC_QUAVERS', 'NO', 'PACMAN', 'PITCHFORK', 'RABBIT', 'ROLLERSKATE', 'SAD', 'SCISSORS', 'SILLY', 'SKULL', 'SMILE', 'SNAKE', 'SQUARE','SQUARE_SMALL" STICKFIGURE', 'SURPRISED', 'SWORD', 'TARGET', 'TORTOISE', 'TRIANGLE', 'TRIANGLE_LEFT', 'TSHIRT', 'UMBRELLA', 'XMAS', 'YES',

Music - import music, dir(music)

'BADDY', 'BA_DING',
'BIRTHDAY', 'BLUES',
'CHASE', 'DADADADUM',
'ENTERTAINER',
'FUNERAL', 'FUNK',
'JUMP_DOWN', 'JUMP_UP',
'NYAN', 'ODE',
'POWER_DOWN',
'POWER_UP', 'PRELUDE',
'PUNCHLINE', 'PYTHON',
'RINGTONE',
'WAWAWAWAA',
'WEDDING', 'get_tempo

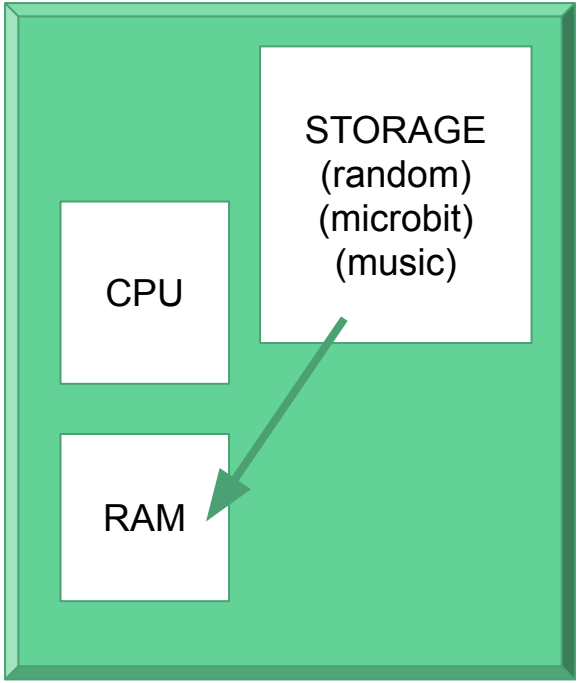
What’s Happening?

```
>>> from microbit import *
>>> display.scroll(“Hi you!”)
```

```
>>> import microbit
>>> microbit.display.scroll(“Hi you!”)
```

Find out what’s inside of a library.

```
>>> dir()
>>> import music
>>> dir(music)
```



Anatomy of micro:bit python program

```
# For-Loop 1
#
# s.case 10/26/20
#
#
```

Comments

```
# Bring in the libraries that you need
from microbit import *
```

Libraries

```
# Execute loop 5 times
for i in range(5):
   display.show(Image.HAPPY)
   sleep(500)
   display.clear()
   sleep(500)
```

Running program

Thonny Editor for Python

Controls

File Edit View Run Tools Help

Files

This computer
/home/steve/microbit/
students

- robot.py
- sound2.py
- speech.py
- temperature1.py
- temperature2.py
- weather_station.py
- while_loop1.py

micro:bit

- 8ball_1.py
- disp_n.py
- eight_ball.py
- for_loop1.py
- main.py

dice.py | [main.py]

Programs on tabs

```
1  #
2  #  For loop demo with display
3  #
4
5  from microbit import *
6  import random
7  display.clear()
8
9
10 while True:
11     for i in range(0, 5):
```

Editing area

Shell | Exception | Program tree | TODO

```
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 Traceback (most recent call last):
   File "<stdin>", line 20, in <module>
 KeyboardInterrupt:
>>>
```

MicroPython (BBC micro:bit) • BBC micro:bit CMSIS-DAP @ /dev/ttyACM0

Reference Material

BBC micro:bit www.microbit.org

IEEE Course Material & Programs
https://github.com/IEEE-BV/microbit

Thonny-Editor Installation:
https://thonny.org/

Adafruit Industries: www.adafruit.com
Sparkfun electronics: www.sparkfun.com

Python:
Micro Python documentation for micro:bit
https://microbit-micropython.readthedocs.io/en/v2-docs/#

Other References:
https://www.w3schools.com/python/default.asp
https://www.python.org/ (advanced full reference for Python)
https://wiki.python.org/moin/BeginnersGuide (still pretty advanced)